# PRACTICA 12

## Crear BD negocios en SQL y migrar datos de MySql

### Objetivo

Crear y configurar mediante Python la base de datos *negocios* en el manejador SQLite, definiendo una estructura equivalente a la existente en MySQL. Posteriormente, desarrollar un proceso de migración que permita extraer los registros almacenados en MySQL e insertarlos en las tablas correspondientes de SQLite, verificando la integridad, correspondencia y consistencia de los datos entre ambos manejadores de bases de datos.

### Objetivos específicos

De manera específica se busca:

- Codificar y ejecutar clase para permite crear BD negocios y sus tablas,
- Instalar librerías necesarias,
- Migrar datos de *MySql* a las tablas de *clientes, productos, ventas, detalleventas,*
- Verificar los registros en las distintas tablas tanto de SQLite como de MySql

### 1. Crear entorno de trabajo

Crear una carpeta sugerida llamada multibase e identificarlo en modo terninal en Visual Studio Code. Crear entonro virtual de trabajo

```
python -m venv .venv
```

Luego

```
.\.venv\Scripts\Activate.ps1
```

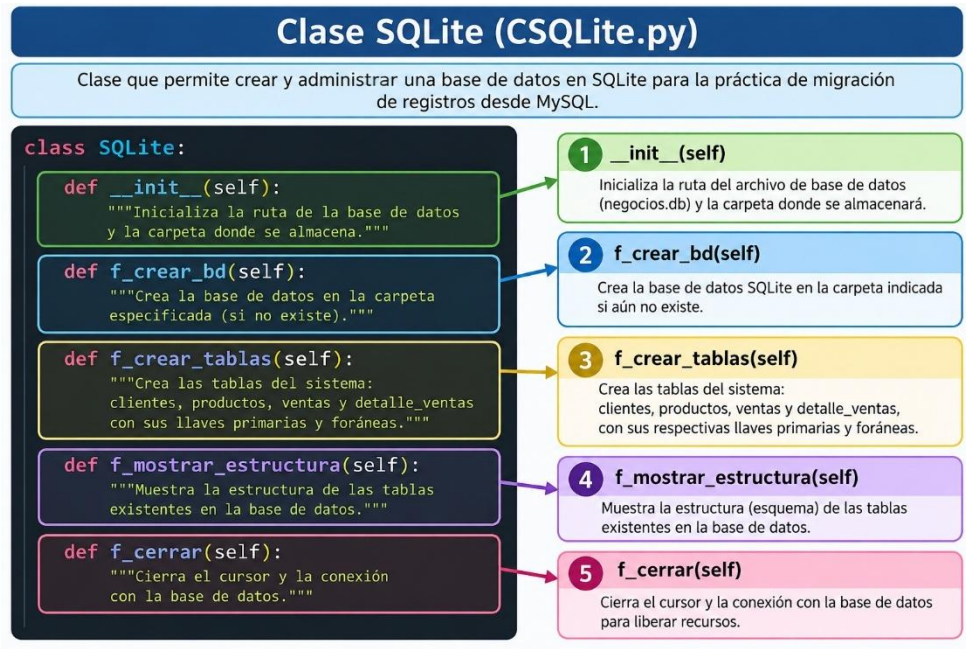
Se deberá activar entorno de trabajo en la carpeta multibase.




## 2. Crear estructura de base de datos negocios

En *Visual Studio Code* cargar el programa Python llamado *CSQLite.py* que crea la clase SQLite que permite crear carpeta datos con la BD negocios, luego crear la estructura de las tablas *con clientes, productos, ventas y detalleventas*; luego muestra la estructura de las tablas y cierra la conexión con SQLite.

La clase *SQLite* con sus métodos:




Aquí el código:

```
import sqlite3
import os
class SQLite:
    def __init__(self):
        self.carpeta = "datos"
        self.bd = os.path.join(self.carpeta, "negocios.db")

        self.conexion = None
        self.cursor = None

    # ------------------------------------------
    # Crear la base de datos
    # ------------------------------------------
    def f_crear_bd(self):
        # Crear carpeta si no existe
        if not os.path.exists(self.carpeta):
            os.makedirs(self.carpeta)

        # Conectarse crea negocios.db si no existe
        self.conexion = sqlite3.connect(self.bd)

        # Crear cursor
        self.cursor = self.conexion.cursor()

        # Activar llaves foráneas
        self.cursor.execute(
            "PRAGMA foreign_keys = ON;"
        )

        print("Base de datos negocios.db creada correctamente")

    def f_crear_tablas(self):
        # ======================================================
        # TABLA: clientes
        # ======================================================

        self.cursor.execute("""
            CREATE TABLE clientes (
                id_cliente CHAR(13) NOT NULL,
                nombre VARCHAR(100) NOT NULL,
                telefono VARCHAR(15),
                correo VARCHAR(100),
                direccion VARCHAR(150),

                CONSTRAINT pk_clientes
```

```
                PRIMARY KEY (id_cliente)
        );
    """)

    # ======================================================
    # TABLA: productos
    # ======================================================

    self.cursor.execute("""
        CREATE TABLE productos (
            id_producto CHAR(20) NOT NULL,
            nombre VARCHAR(100) NOT NULL,
            descripcion VARCHAR(200),
            precio DECIMAL(10,2) NOT NULL,
            existencia INTEGER NOT NULL,

            CONSTRAINT pk_productos
                PRIMARY KEY (id_producto)
        );
    """)


    # ======================================================
    # TABLA: ventas
    # ======================================================

    self.cursor.execute("""
        CREATE TABLE ventas (
            id_venta INTEGER PRIMARY KEY AUTOINCREMENT,
            fecha TIMESTAMP NOT NULL
                DEFAULT CURRENT_TIMESTAMP,
            id_cliente CHAR(13) NOT NULL,
            total DECIMAL(10,2),

            CONSTRAINT fk_ventas_clientes
                FOREIGN KEY (id_cliente)
                REFERENCES clientes(id_cliente)
        );
    """)

    # ======================================================
    # TABLA: detalle_ventas
    # ======================================================

    self.cursor.execute("""
        CREATE TABLE detalle_ventas (
```

```
            id_detalle INTEGER PRIMARY KEY AUTOINCREMENT,
            id_venta INTEGER NOT NULL,
            id_producto CHAR(20) NOT NULL,
            cantidad INTEGER NOT NULL,
            precio_unitario DECIMAL(10,2) NOT NULL,
            subtotal DECIMAL(10,2),

            CONSTRAINT fk_detalle_ventas
                FOREIGN KEY (id_venta)
                REFERENCES ventas(id_venta),

            CONSTRAINT fk_detalle_productos
                FOREIGN KEY (id_producto)
                REFERENCES productos(id_producto)
        );
    """)

    # ======================================================
    # GUARDAR CAMBIOS
    # ======================================================

    self.conexion.commit()

    print("Tablas creadas correctamente")

def f_mostrar_estructura(self):

    tablas = [
        "clientes",
        "productos",
        "ventas",
        "detalle_ventas"
    ]

    for tabla in tablas:

        print("\nTABLA:", tabla)
        print("-" * 50)

        self.cursor.execute(
            f"PRAGMA table_info({tabla});"
        )

        columnas = self.cursor.fetchall()

        for columna in columnas:
```

```
                print(columna)

    def f_cerrar(self):

        if self.conexion:
            self.conexion.close()

            print("Conexión SQLite cerrada")
```

Ejecutar en modo terminal de *Visual Studio Code* el programa *probar CSQLite* que ejecuta los métodos para crear la base de datos, las tablas y mostrar la estructura de los datos. Se debe observar que ya se tiene la estructura.

```
python '.\probar CSQLite.py'
```

Aquí el código:

```
from CSQLite import SQLite

# Crear objeto
sqlite = SQLite()

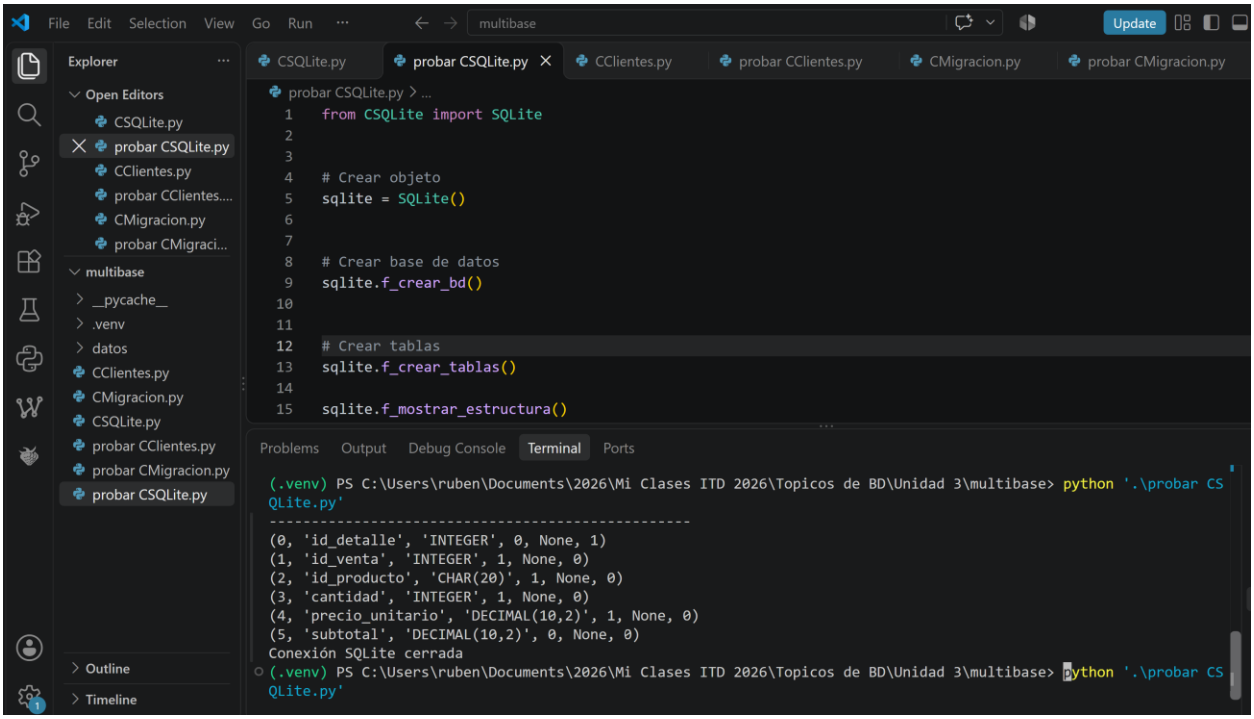
# Crear base de datos
sqlite.f_crear_bd()

# Crear tablas
sqlite.f_crear_tablas()

sqlite.f_mostrar_estructura()

# Cerrar conexión
sqlite.f_cerrar()
```

Se observa la siguiente salida con los atributos de cada tabla:

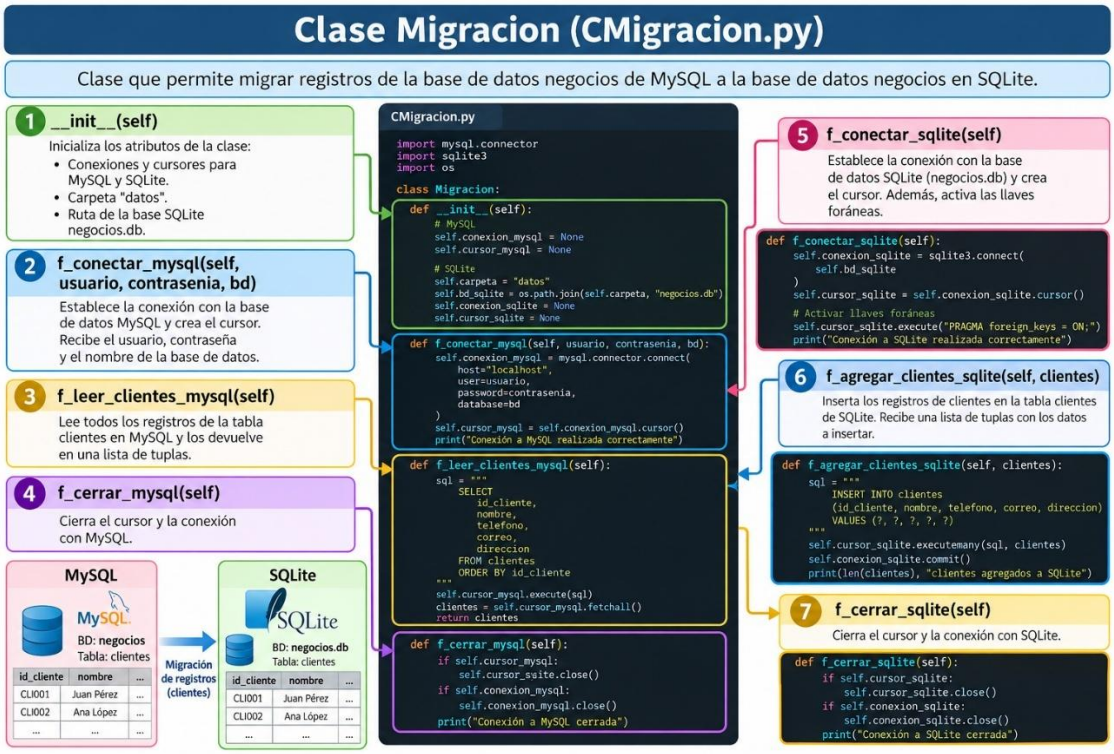



### 3. Migrar datos de clientes de MySql a SQLite

Para migrar los datos se requiere instalar la libería que hace la conexión a MySql.

```
pip install mysql-connector-python
```

Ahora se deberá cargar el archivo *CMigracion.py* que contiene la clase *Migracion* con los métodos para migrar datos de la tabla clientes de *MySql* a la tabla *clientes* de *SQLite*. Esto garantizará tener los mismos registros y valores en cada una de las tablas de ambos bases de datos de los dos manejadores de BD, tanto de *MySql* como de *SQLite*.




Aquí el código con los métodos para migrar los datos de clientes:

```
import mysql.connector
import sqlite3
import os

class Migracion:

    def __init__(self):
        # MySQL
        self.conexion_mysql = None
        self.cursor_mysql = None
        # SQLite
        self.carpeta = "datos"
        self.bd_sqlite = os.path.join(
            self.carpeta,
            "negocios.db"
        )
        self.conexion_sqlite = None
        self.cursor_sqlite = None

    # ------------------------------------------------
    # CONECTAR A MYSQL
    # ------------------------------------------------
```

```
    def f_conectar_mysql(self, usuario, contrasenia, bd):

        self.conexion_mysql = mysql.connector.connect(
            host="localhost",
            user=usuario,
            password=contrasenia,
            database=bd
        )

        self.cursor_mysql = self.conexion_mysql.cursor()

        print("Conexión a MySQL realizada correctamente")

    # ------------------------------------------------
    # LEER CLIENTES DE MYSQL
    # ------------------------------------------------
    def f_leer_clientes_mysql(self):

        sql = """
            SELECT
                id_cliente,
                nombre,
                telefono,
                correo,
                direccion
            FROM clientes
            ORDER BY id_cliente
        """

        self.cursor_mysql.execute(sql)

        clientes = self.cursor_mysql.fetchall()

        return clientes

    # ------------------------------------------------
    # CERRAR MYSQL
    # ------------------------------------------------
    def f_cerrar_mysql(self):

        if self.cursor_mysql:
            self.cursor_mysql.close()

        if self.conexion_mysql:
            self.conexion_mysql.close()
```

```
        print("Conexión a MySQL cerrada")

    # ------------------------------------------------
    # CONECTAR A SQLITE
    # ------------------------------------------------
    def f_conectar_sqlite(self):

        self.conexion_sqlite = sqlite3.connect(
            self.bd_sqlite
        )

        self.cursor_sqlite = (
            self.conexion_sqlite.cursor()
        )

        # Activar llaves foráneas
        self.cursor_sqlite.execute(
            "PRAGMA foreign_keys = ON;"
        )

        print(
            "Conexión a SQLite realizada correctamente"
        )

        # ------------------------------------------------
    # AGREGAR CLIENTES A SQLITE
    # ------------------------------------------------
    def f_agregar_clientes_sqlite(self, clientes):

        sql = """
            INSERT INTO clientes
            (
                id_cliente,
                nombre,
                telefono,
                correo,
                direccion
            )
            VALUES (?, ?, ?, ?, ?)
        """

        self.cursor_sqlite.executemany(
            sql,
            clientes
        )
```

```
        self.conexion_sqlite.commit()

        print(
            len(clientes),
            "clientes agregados a SQLite"
        )
    # ------------------------------------------------
    # CERRAR SQLITE
    # ------------------------------------------------
    def f_cerrar_sqlite(self):

        if self.cursor_sqlite:
            self.cursor_sqlite.close()

        if self.conexion_sqlite:
            self.conexion_sqlite.close()

        print("Conexión a SQLite cerrada")
```

Ahora, se debe ejecutar por medio de *Python*, creando un objeto de la clase *Migracion*, para conectarse a *MySql*; dar lectura de los datos clientes; insertar los registros en la tabla de clientes de *SQLite*; cerrar las conexiones tanto de *MySql* como de *SQLite*.

Ejecutar el archivo probar *CMigracion.py* desde modo terminal en *Visual Studio Code* el cual hace la migración de los datos clientes.

La conexión al servicio local de *MySql* se hace con el usuario *'rpizarro' con* contraseña *'rpizarro'* a la *BD 'negocios'.*

```
from CMigracion import Migracion

migracion = Migracion()

# ============================================
# 1. LEER CLIENTES DE MYSQL
# ============================================
migracion.f_conectar_mysql("rpizarro", "rpizarro", "negocios")

clientes = migracion.f_leer_clientes_mysql()

print("\nClientes encontrados en MySQL:")
```

```
for cliente in clientes:
    print(cliente)

migracion.f_cerrar_mysql()

# ============================================
# 2. MIGRAR CLIENTES A SQLITE
# ============================================

migracion.f_conectar_sqlite()

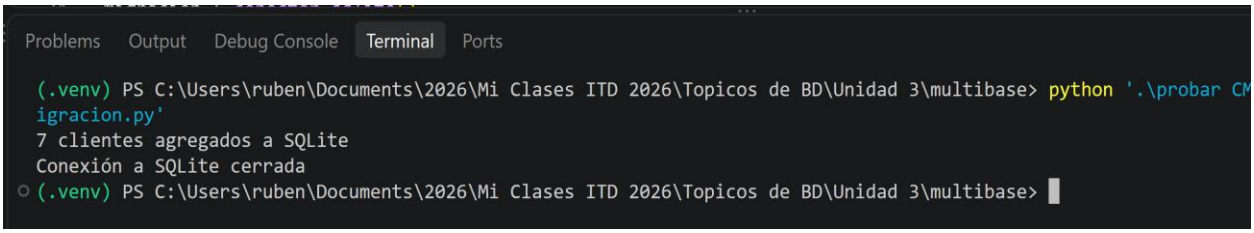
migracion.f_agregar_clientes_sqlite(
    clientes
)

migracion.f_cerrar_sqlite()
```

Se ejecuta:

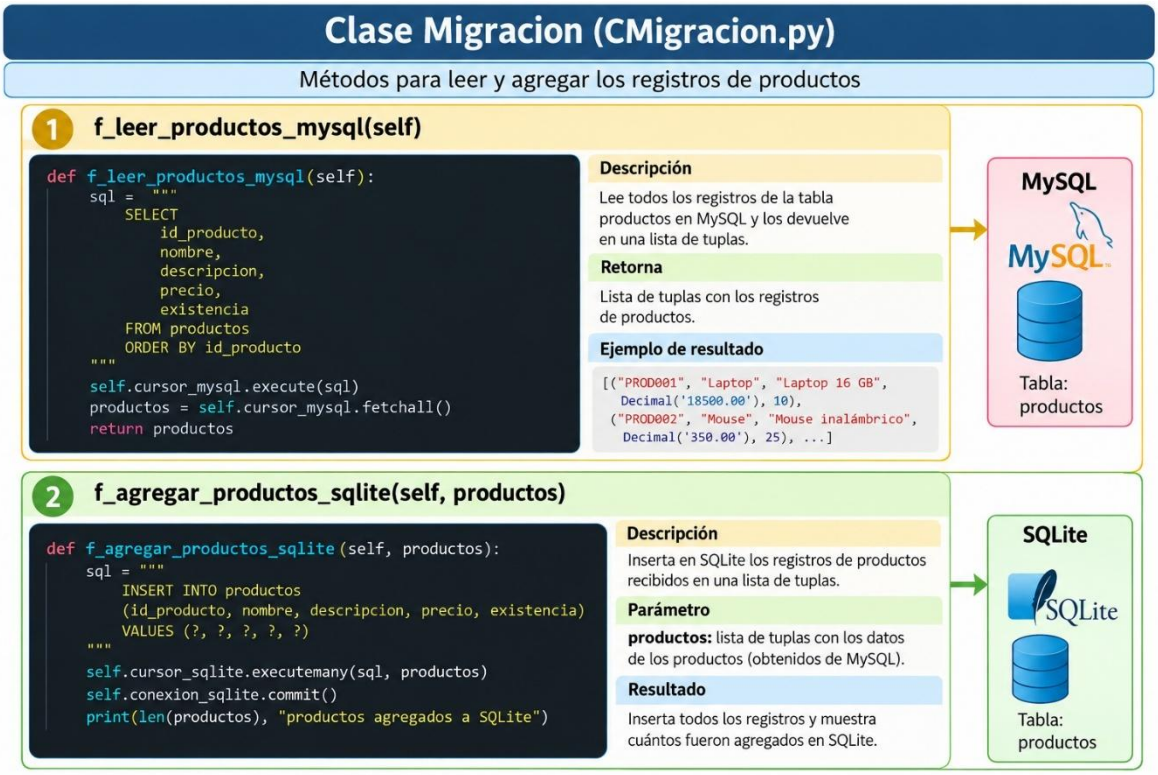
```
python '.\probar CMigracion.py'
```

y la salida algo similar a esta:




## 4. Migrar datos de productos de MySql a SQLite

Agregar estos dos métodos a la clase *Migracion* al archivo *CMigracion.py*:




```
# ------------------------------------------------
# LEER PRODUCTOS DE MYSQL
# ------------------------------------------------
def f_leer_productos_mysql(self):

    sql = """
        SELECT
            id_producto,
            nombre,
            descripcion,
            precio,
            existencia
        FROM productos
        ORDER BY id_producto
    """

    self.cursor_mysql.execute(sql)

    productos = self.cursor_mysql.fetchall()

    return productos

def f_agregar_productos_sqlite(self, productos):

    productos_sqlite = []
```

```
        for producto in productos:

            id_producto = producto[0]
            nombre = producto[1]
            descripcion = producto[2]
            precio = float(producto[3]) # Conversion de decimal a float
            existencia = producto[4]

            productos_sqlite.append(
                (
                    id_producto,
                    nombre,
                    descripcion,
                    precio,
                    existencia
                )
            )

        sql = """
            INSERT INTO productos
            (
                id_producto,
                nombre,
                descripcion,
                precio,
                existencia
            )
            VALUES (?, ?, ?, ?, ?)
        """

        self.cursor_sqlite.executemany(
            sql,
            productos_sqlite
        )

        self.conexion_sqlite.commit()

        print(
            len(productos_sqlite),
            "productos agregados a SQLite"
        )
```

Ejecutar nuevamente el archivo *probar Migracion.py* integrando las líneas de código que hacen la migración de los datos de productos:

```
from CMigracion import Migracion

migracion = Migracion()

# ============================================
# 1. LEER CLIENTES DE MYSQL
# ============================================

migracion.f_conectar_mysql("rpizarro", "rpizarro", "negocios")

clientes = migracion.f_leer_clientes_mysql()
productos = migracion.f_leer_productos_mysql()

print("\nClientes encontrados en MySQL:")

for cliente in clientes:
    print(cliente)

for producto in productos:
    print(producto)

migracion.f_cerrar_mysql()

# ============================================
# 2. MIGRAR CLIENTES A SQLITE
# ============================================

migracion.f_conectar_sqlite()

migracion.f_agregar_clientes_sqlite(clientes)

# ============================================
# 3. MIGRAR PRODUCTIOS A SQLITE
# ============================================

migracion.f_agregar_productos_sqlite(productos)

migracion.f_cerrar_sqlite()
```

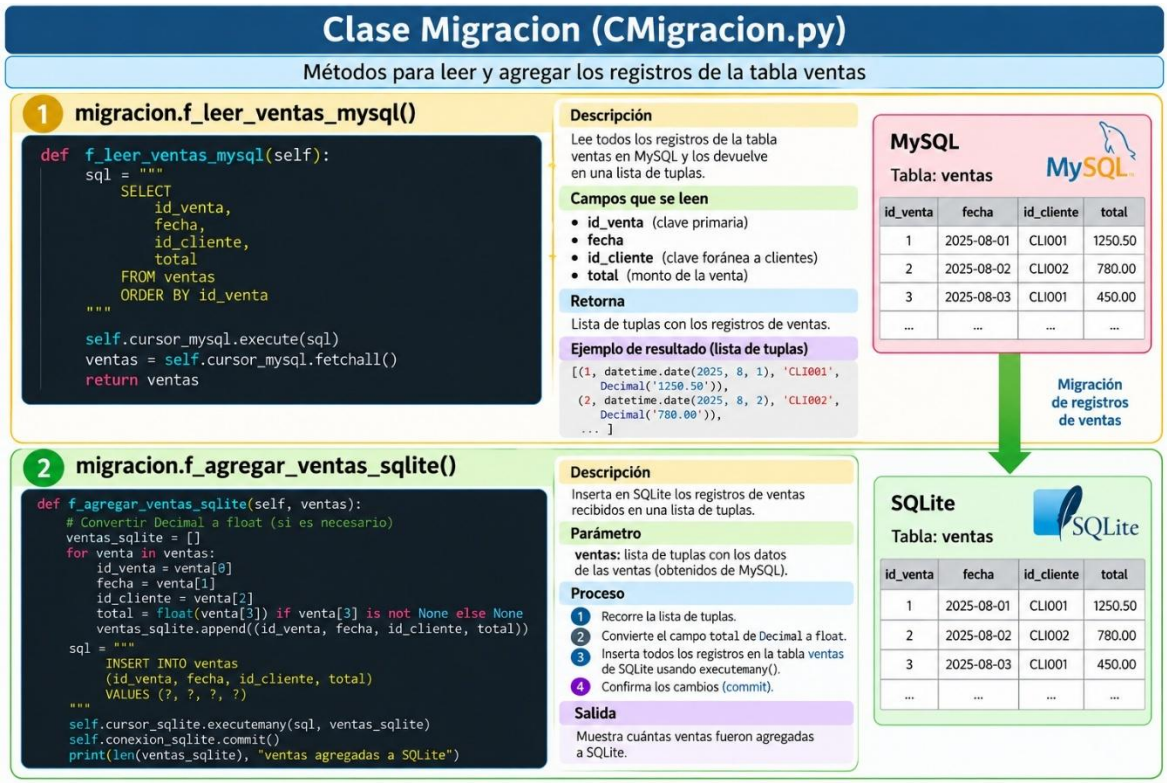
Aquí la pantalla de salida similar a la siguiente:

```
('MESAOF', 'Mesa de oficina ejecutiva', 'Mesa de oficina ejecutiva, mesa y silla', Decimal('2500.00'), 30)
('PROD0000000000000001', 'Teclado', 'Teclado USB', Decimal('333.33'), 50)
('PROD0000000000000002', 'Mouse', 'Mouse optico USB', Decimal('200.00'), 60)
('PROD0000000000000003', 'Monitor', 'Monitor LED 24 pulgadas', Decimal('3200.00'), 20)
('PROD0000000000000004', 'Memoria USB', 'Memoria USB 64 GB', Decimal('250.00'), 100)
('PROD0000000000000005', 'Audifonos', 'Audifonos con microfono', Decimal('600.00'), 40)
Conexión a MySQL cerrada
Conexión a SQLite realizada correctamente
Traceback (most recent call last):
```

## 5. Migrar datos de ventas de MySql a SQLite

Ahora se presenta el proceso para migrar los daros de ventas similar a los procesos de clientes y productos.

Primero leer los datos de *ventas* de *MySql* y luego agregar a la tabla *ventas* de *SQLite*.




Se modifica la clase *Migracion* para agregar los métodos *f_leer_ventas_mysql()* y *f_agregar_ventas_sqlite()*.

```
    # ------------------------------------------------
    # LEER VENTAS DE MYSQL
    # ------------------------------------------------
    def f_leer_ventas_mysql(self):

        sql = """
            SELECT
                id_venta,
                fecha,
                id_cliente,
                total
            FROM ventas
            ORDER BY id_venta
        """

        self.cursor_mysql.execute(sql)

        ventas = self.cursor_mysql.fetchall()

        return ventas

    # ------------------------------------------------
    # AGREGAR VENTAS A SQLITE
    # ------------------------------------------------
    def f_agregar_ventas_sqlite(self, ventas):

        ventas_sqlite = []

        for venta in ventas:

            id_venta = venta[0]
            fecha = venta[1]
            id_cliente = venta[2]

            # Convertir Decimal de MySQL a float
            if venta[3] is not None:
                total = float(venta[3])
            else:
                total = None

            ventas_sqlite.append(
                (
                    id_venta,
                    fecha,
                    id_cliente,
                    total
                )
            )

        sql = """
```

```
            INSERT INTO ventas
            (
                id_venta,
                fecha,
                id_cliente,
                total
            )
            VALUES (?, ?, ?, ?)
        """

        self.cursor_sqlite.executemany(
            sql,
            ventas_sqlite
        )

        self.conexion_sqlite.commit()

        print(
            len(ventas_sqlite),
            "ventas agregadas a SQLite"
        )
```

Ahora ejecutar el archivo *probar Migracion.py* con las instrucciones particulares de la migración de solo las *ventas*:

```
from CMigracion import Migracion

migracion = Migracion()

migracion.f_conectar_mysql("rpizarro", "rpizarro", "negocios")

clientes = migracion.f_leer_clientes_mysql()

productos = migracion.f_leer_productos_mysql()

ventas = migracion.f_leer_ventas_mysql()

print("\nVentas en MySQL:")

for venta in ventas:

    print(venta)

migracion.f_cerrar_mysql()

migracion.f_conectar_sqlite()

# ============================================

# 3. MIGRAR VENTAS A SQLITE

# ============================================

migracion.f_agregar_ventas_sqlite(ventas)

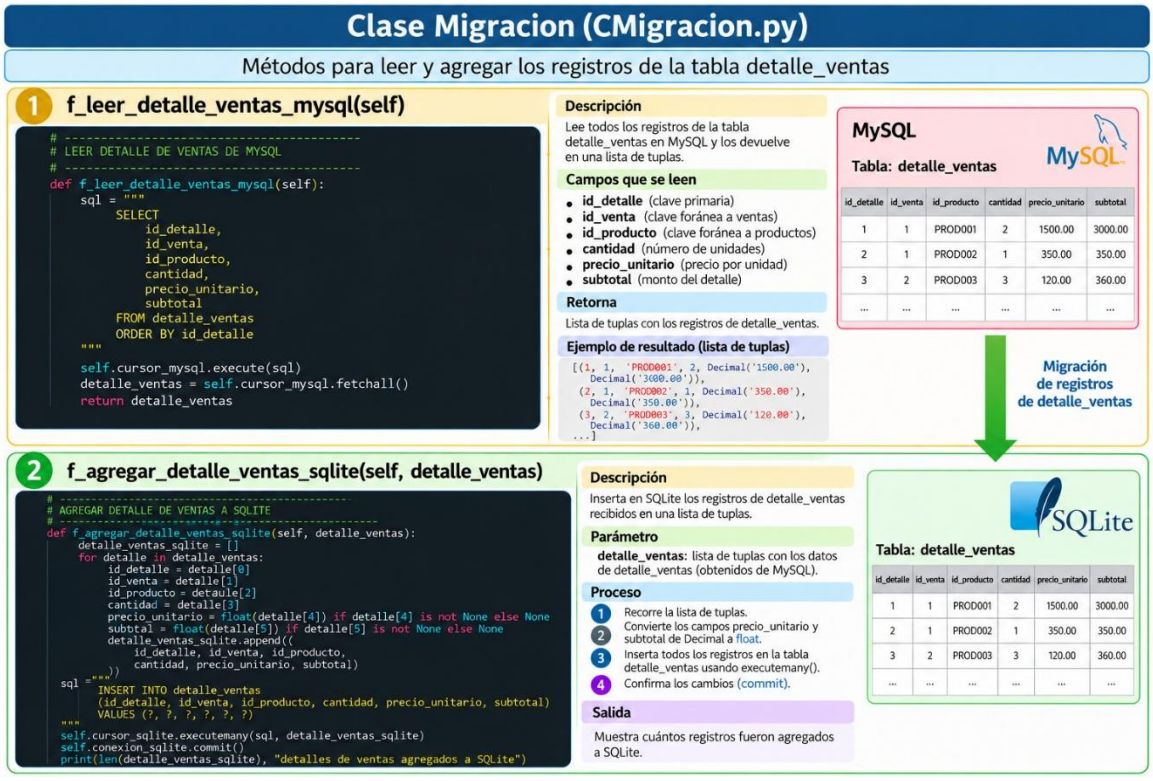
migracion.f_cerrar_sqlite()
```

La salida de la migración de ventas se presenta en una pantalla similar a esta:

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\Unidad 3\multibase> & "c:\Users\ruben\
ocuments\2026\Mi Clases ITD 2026\Topicos de BD\Unidad 3\multibase\.venv\Scripts\python.exe" "c:/Users/ruben/Docu
ents/2026/Mi Clases ITD 2026/Topicos de BD/Unidad 3/multibase/probar CMigracion.py"
Conexión a MySQL realizada correctamente
(1, datetime.datetime(2026, 9, 7, 18, 13, 7), 'CLI0000000001', Decimal('900.00'))
(2, datetime.datetime(2026, 9, 7, 18, 13, 7), 'CLI0000000002', Decimal('3700.00'))
(3, datetime.datetime(2026, 9, 7, 18, 13, 7), 'CLI0000000003', Decimal('1250.00'))
(4, datetime.datetime(2026, 9, 7, 18, 13, 7), 'CLI0000000004', Decimal('1550.00'))
(5, datetime.datetime(2026, 9, 7, 18, 13, 7), 'CLI0000000005', Decimal('3900.00'))
Conexión a SQLite realizada correctamente
5 ventas agregadas a SQLite
Conexión a SQLite cerrada
```

## 6. Migrar datos de detalle_ventas de MySql a SQLite

Ahora se agregan los métodos *f_leer_detalle_ventas_mysql()* para leer los datos de la tabla *detalle_ventas* y *f_agregar_detalle_ventas_sqlite()* para agregar registros a la tabla *detalle_ventas.*




Se presenta el código de los métodos que se integran a la clase *Migracion*:

```
    # ------------------------------------------------
    # LEER DETALLE DE VENTAS DE MYSQL
    # ------------------------------------------------
    def f_leer_detalle_ventas_mysql(self):

        sql = """
            SELECT
                id_detalle,
                id_venta,
                id_producto,
                cantidad,
                precio_unitario,
                subtotal
            FROM detalle_ventas
            ORDER BY id_detalle
        """

        self.cursor_mysql.execute(sql)

        detalle_ventas = self.cursor_mysql.fetchall()

        return detalle_ventas

    # ------------------------------------------------
    # AGREGAR DETALLE DE VENTAS A SQLITE
    # ------------------------------------------------
    def f_agregar_detalle_ventas_sqlite(
        self,
        detalle_ventas
    ):

        detalle_ventas_sqlite = []

        for detalle in detalle_ventas:

            id_detalle = detalle[0]
            id_venta = detalle[1]
            id_producto = detalle[2]
            cantidad = detalle[3]

            # Convertir DECIMAL de MySQL a float
            if detalle[4] is not None:
                precio_unitario = float(detalle[4])
            else:
                precio_unitario = None

            # Convertir DECIMAL de MySQL a float
            if detalle[5] is not None:
                subtotal = float(detalle[5])
            else:
```

```
                subtotal = None

            detalle_ventas_sqlite.append(
                (
                    id_detalle,
                    id_venta,
                    id_producto,
                    cantidad,
                    precio_unitario,
                    subtotal
                )
            )

        sql = """
            INSERT INTO detalle_ventas
            (
                id_detalle,
                id_venta,
                id_producto,
                cantidad,
                precio_unitario,
                subtotal
            )
            VALUES (?, ?, ?, ?, ?, ?)
        """

        self.cursor_sqlite.executemany(
            sql,
            detalle_ventas_sqlite
        )

        self.conexion_sqlite.commit()

        print(
            len(detalle_ventas_sqlite),
            "detalles de ventas agregados a SQLite"
        )
```

Al ejecutar el archivo *probar Migracion.py* se migran los registros de *detalle_ventas*.

Se debe observar un apantalla similar a la siguiente en la que se observa todos los datos migrados:

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\Unidad 3\multibase> & "c:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\Unidad 3\multibase\.venv\Scripts\python.exe" "c:/Users/ruben/Documents/2026/Mi Clases ITD 2026/Topicos de BD/Unidad 3/multibase/probar CMigracion.py"
(3, 2, 'PROD0000000000000003', 1, Decimal('3200.00'), Decimal('3200.00'))
(4, 2, 'PROD0000000000000004', 2, Decimal('250.00'), Decimal('500.00'))
(5, 3, 'PROD0000000000000005', 1, Decimal('600.00'), Decimal('600.00'))
(6, 3, 'PROD0000000000000002', 2, Decimal('200.00'), Decimal('400.00'))
(7, 3, 'PROD0000000000000004', 1, Decimal('250.00'), Decimal('250.00'))
(8, 4, 'PROD0000000000000001', 1, Decimal('350.00'), Decimal('350.00'))
(9, 4, 'PROD0000000000000005', 2, Decimal('600.00'), Decimal('1200.00'))
(10, 5, 'PROD0000000000000003', 1, Decimal('3200.00'), Decimal('3200.00'))
(11, 5, 'PROD0000000000000002', 1, Decimal('200.00'), Decimal('200.00'))
(12, 5, 'PROD0000000000000004', 2, Decimal('250.00'), Decimal('500.00'))
Conexión a MySQL cerrada
Conexión a SQLite realizada correctamente
7 clientes agregados a SQLite
6 productos agregados a SQLite
5 ventas agregadas a SQLite
12 detalles de ventas agregados a SQLite
```

## 7. Archivos en github.com de la PRATICA 12

En la dirección *github.com* siguiente se encuentran los archivos que se deben ejecutar para hacer la migración completa.

topicos-selectos-de-bd/PRACTICA 12 at main · rpizarrog/topicos-selectos-de-bd

Al final se debe tener las mismas tablas, los mismos registros y los mimos valores en *MySql* como en *SQLite*.

FIN DE LA PRACTICA